

MY SQL QUERIES

COFFEE SHOP SALES PROJECT

- **CREATE coffee_shop_sales_db DATABASE**

```
create database coffee_shop_sales_db;
```

- **DATA TYPES OF DIFFERENT COLUMNS BEFORE CLEANING**

```
describe coffee_shop_sales;
```

Result Grid Filter Rows:				
	Field	Type	Null	Key
►	transaction_id	text	YES	
	transaction_date	text	YES	
	transaction_time	text	YES	
	transaction_qty	text	YES	
	store_id	text	YES	
	store_location	varchar(100)	YES	
	product_id	text	YES	
	unit_price	decimal(5,2)	YES	
	product_category	varchar(50)	YES	
	product_type	varchar(100)	YES	
	product_detail	varchar(255)	YES	

- **CONVERT DATE (transaction_date) COLUMN TO PROPER DATE FORMAT**

```
UPDATE coffee_shop_sales
```

```
SET transaction_date = STR_TO_DATE (transaction_date, '%d-%m-%Y');
```

- **ALTER DATE (transaction_date) COLUMN TO DATE DATA TYPE**

```
ALTER TABLE coffee_shop_sales
```

```
MODIFY COLUMN transaction_date DATE;
```

- **ALTER DATE (transaction_id, store_id, product_id, transaction_qty) COLUMNS TO INTEGER DATA TYPE**

- ALTER TABLE coffee_shop_sales
MODIFY COLUMN transaction_id INT;
- ALTER TABLE coffee_shop_sales
MODIFY COLUMN store_id INT;
- ALTER TABLE coffee_shop_sales
MODIFY COLUMN product_id INT;
- ALTER TABLE coffee_shop_sales
MODIFY COLUMN transaction_qty INT;

- **CONVERT TIME (transaction_time) COLUMN TO PROPER DATE FORMAT**

UPDATE coffee_shop_sales

SET transaction_time = STR_TO_DATE (transaction_time, '%H:%i:%s');

- **ALTER TIME (transaction_time) COLUMN TO DATE DATA TYPE**

ALTER TABLE coffee_shop_sales

MODIFY COLUMN transaction_time TIME;

- **DATA TYPES OF DIFFERENT COLUMNS AFTER CLEANING**

DESCRIBE coffee_shop_sales;

Result Grid				
Filter Rows:				
	Field	Type	Null	
►	transaction_id	int	YES	
	transaction_date	date	YES	
	transaction_time	time	YES	
	transaction_qty	int	YES	
	store_id	int	YES	
	store_location	varchar(100)	YES	
	product_id	int	YES	
	unit_price	decimal(5,2)	YES	
	product_category	varchar(50)	YES	
	product_type	varchar(100)	YES	
	product_detail	varchar(255)	YES	

KPI's REQUIREMENTS

(1) TOTAL SALES ANALYSIS

a. Total Sales For Month

```
SELECT CONCAT(ROUND(SUM(unit_price * transaction_qty)/1000), "K") as Total_Sales
FROM coffee_shop_sales
WHERE MONTH(transaction_date) = 5; -- for month of (CM-May)
```

Result Grid	
	Total_sales
▶	157K

b. Total Sales KPI - Mom Difference And Mom Growth

```
SELECT
    MONTH(transaction_date) AS Month ,
    ROUND(SUM(unit_price * transaction_qty)) AS Total_Sales,
    ROUND(SUM(unit_price * transaction_qty) - LAG(SUM(unit_price * transaction_qty), 1)
    OVER (ORDER BY MONTH(transaction_date))) AS Moth_Sales_Diff,
    ROUND((SUM(unit_price * transaction_qty) - LAG(SUM(unit_price * transaction_qty), 1)
    OVER (ORDER BY MONTH(transaction_date))) / LAG(SUM(unit_price * transaction_qty), 1)
    OVER (ORDER BY MONTH(transaction_date)) * 100,1) AS MoM_Increase_Percentage
FROM
    coffee_shop_sales
WHERE
    MONTH(transaction_date) IN (4, 5) -- for months of April and May
GROUP BY
    MONTH(transaction_date)
ORDER BY
    MONTH(transaction_date);
```

Result Grid		Filter Rows:	Export:	Wrap
	Month	Total_sales	Month_Sales_Diff	MoM_Increase_Percentage
▶	4	118941	NULL	NULL
	5	156728	37787	31.8

Explanation

SELECT clause:

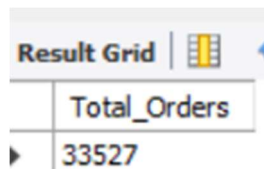
- **MONTH(transaction_date) AS Month:** Extracts the month from the transaction_date column and renames it as month.
- **ROUND(SUM(unit_price * transaction_qty)) AS Total_Sales:** Calculates the total sales by multiplying unit_price and transaction_qty, then sums the result for each month. The ROUND function rounds the result to the nearest integer.
- **(SUM(unit_price * transaction_qty) - LAG(SUM(unit_price * transaction_qty), 1) OVER (ORDER BY MONTH(transaction_date))) / LAG(SUM(unit_price * transaction_qty), 1) OVER (ORDER BY MONTH(transaction_date)) * 100 AS MoM_Increase_Percentage** with the functions used:
 - **SUM(unit_price * transaction_qty):** This calculates the total sales for the current month. It multiplies the unit_price by the transaction_qty for each transaction and then sums up these values.
 - **LAG(SUM(unit_price * transaction_qty), 1) OVER (ORDER BY MONTH(transaction_date)):** This function retrieves the value of the total sales for the previous month. It uses the LAG window function to get the value of the SUM(unit_price * transaction_qty) from the previous row (previous month) ordered by the transaction_date.
 - **(SUM(unit_price * transaction_qty) - LAG(SUM(unit_price * transaction_qty), 1) OVER (ORDER BY MONTH(transaction_date))):** This part calculates the difference between the total sales of the current month and the total sales of the previous month.
 - **LAG(SUM(unit_price * transaction_qty), 1) OVER (ORDER BY MONTH(transaction_date)):** This function retrieves the value of the total sales for the previous month again. It's used in the denominator to calculate the percentage increase or decrease.
 - **(SUM(unit_price * transaction_qty) - LAG(SUM(unit_price * transaction_qty), 1) OVER (ORDER BY MONTH(transaction_date))) / LAG(SUM(unit_price * transaction_qty), 1) OVER (ORDER BY MONTH(transaction_date)):** This calculates the ratio of the difference in sales between the current and previous months to the total sales of the previous month. It represents the percentage increase or decrease in sales compared to the previous month.
 - **100:** This part multiplies the ratio by 100 to convert it to a percentage.
- **FROM clause:**
coffee_shop_sales: Specifies the table from which data is being selected.
- **WHERE clause:**
MONTH(transaction_date) IN (4, 5): Filters the data to include only transactions from April and May.
- **GROUP BY clause:**
MONTH(transaction_date): Groups the results by month.
- **ORDER BY clause:**

MONTH(transaction_date): Orders the results by month.

(2) TOTAL ORDER ANALYSIS

a. Total Order For Month

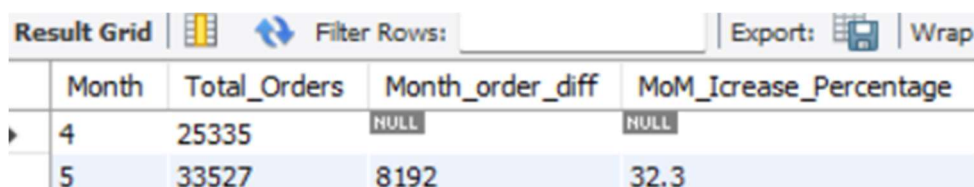
```
SELECT COUNT (transaction_id) as Total_Orders
FROM coffee_shop_sales
WHERE MONTH (transaction_date) = 5; -- for month of (CM-May)
```



Total_Orders
33527

b. Total Orders KPI - Mom Difference And Mom Growth

```
SELECT
    MONTH(transaction_date) AS Month,
    ROUND(COUNT(transaction_id)) AS Total_Orders,
    COUNT(transaction_id) - LAG(COUNT(transaction_id), 1)
    OVER (ORDER BY MONTH(transaction_date)) AS Month_Order_Diff,
    ROUND((COUNT(transaction_id) - LAG(COUNT(transaction_id), 1)
    OVER (ORDER BY MONTH(transaction_date))) / LAG(COUNT(transaction_id), 1)
    OVER (ORDER BY MONTH(transaction_date)) * 100,1) AS MoM_Increase_Percentage
FROM
    coffee_shop_sales
WHERE
    MONTH(transaction_date) IN (4, 5) -- for April and May
GROUP BY
    MONTH(transaction_date)
ORDER BY
    MONTH(transaction_date);
```



Month	Total_Orders	Month_order_diff	MoM_Icrease_Percentage
4	25335	NULL	NULL
5	33527	8192	32.3

(3) TOTAL QUANTITY SOLD ANALYSIS

a. Total Quantity Sold For Month

```
SELECT SUM(transaction_qty) as Total_Quantity_Sold
FROM coffee_shop_sales
WHERE MONTH(transaction_date) = 5; -- for month of (CM-May)
```

Result Grid	
Total_Quantity_Sold	
▶	48233

b. Total Quantity Sold KPI - Mom Difference And Mom Growth

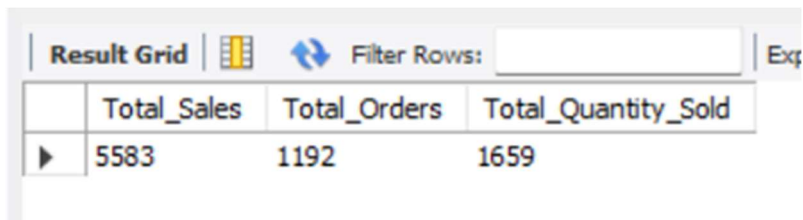
```
SELECT
    MONTH(transaction_date) AS Month,
    ROUND(SUM(transaction_qty)) AS Total_Quantity_Sold,
    (SUM(transaction_qty) - LAG(SUM(transaction_qty), 1)
    OVER (ORDER BY MONTH(transaction_date))) as Month_Sold_Quantiy_Diff,
    ROUND((SUM(transaction_qty) - LAG(SUM(transaction_qty), 1)
    OVER (ORDER BY MONTH(transaction_date))) / LAG(SUM(transaction_qty), 1)
    OVER (ORDER BY MONTH(transaction_date)) * 100,1) AS MoM_Increase_Percentage
FROM
    coffee_shop_sales
WHERE
    MONTH(transaction_date) IN (4, 5) -- for April and May
GROUP BY
    MONTH(transaction_date)
ORDER BY
    MONTH(transaction_date);
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
month	total_quantity_sold	Month_Sold_Quantiy_Diff	MoM_Increase_Percentage	
▶ 4	36469	NULL	NULL	
5	48233	11764	32.3	

CHARTS REQUIREMENTS

- For Calendar Table – Daily Sales, Quantity And Total Orders

```
SELECT
    ROUND(SUM(unit_price * transaction_qty)) AS Total_Sales,
    COUNT(transaction_id) AS Total_Orders,
    SUM(transaction_qty) AS Total_Quantity_Sold
FROM
    coffee_shop_sales
WHERE
    transaction_date = '2023-05-18'; -- For 18 May 2023
```

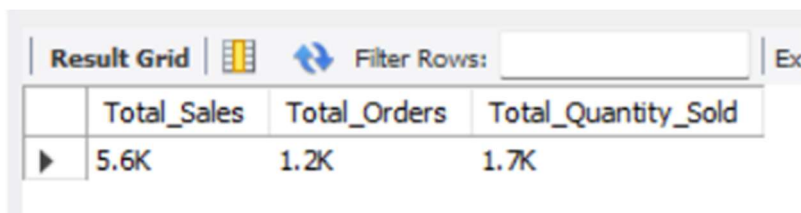


The screenshot shows a 'Result Grid' window with a 'Filter Rows' input field. The grid contains one row of data with three columns: 'Total_Sales', 'Total_Orders', and 'Total_Quantity_Sold'. The values are 5583, 1192, and 1659 respectively.

	Total_Sales	Total_Orders	Total_Quantity_Sold
▶	5583	1192	1659

If you want to get exact Rounded off values then use below query to get the result:

```
SELECT
    CONCAT(ROUND(SUM(unit_price * transaction_qty) / 1000, 1), 'K') AS Total_Sales,
    CONCAT(ROUND(COUNT(transaction_id) / 1000, 1), 'K') AS Total_Orders,
    CONCAT(ROUND(SUM(transaction_qty) / 1000, 1), 'K') AS Total_Quantity_Sold
FROM
    coffee_shop_sales
WHERE
    transaction_date = '2023-05-18'; -- For 18 May 2023
```

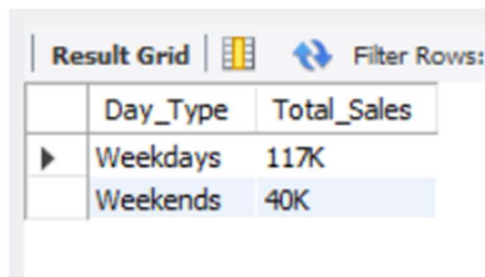


The screenshot shows a 'Result Grid' window with a 'Filter Rows' input field. The grid contains one row of data with three columns: 'Total_Sales', 'Total_Orders', and 'Total_Quantity_Sold'. The values are 5.6K, 1.2K, and 1.7K respectively.

	Total_Sales	Total_Orders	Total_Quantity_Sold
▶	5.6K	1.2K	1.7K

- Sales By Weekday / Weekend

```
SELECT
    CASE
        WHEN DAYOFWEEK (transaction_date) IN (1, 7) THEN 'Weekends'
        ELSE 'Weekdays'
    END AS Day_Type,
    CONCAT (ROUND (SUM (unit_price * transaction_qty)/1000),'K') AS Total_Sales
FROM
    coffee_shop_sales
WHERE
    MONTH (transaction_date) = 5 -- For May
GROUP BY
    CASE
        WHEN DAYOFWEEK (transaction_date) IN (1, 7) THEN 'Weekends'
        ELSE 'Weekdays'
    END;
```

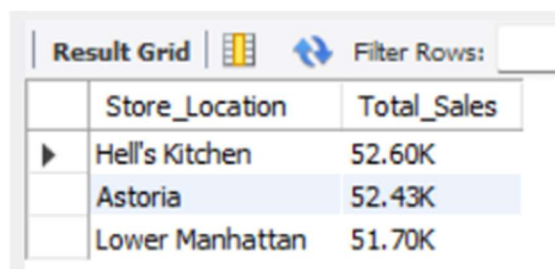


The screenshot shows a 'Result Grid' with two columns: 'Day_Type' and 'Total_Sales'. The data is as follows:

Day_Type	Total_Sales
Weekdays	117K
Weekends	40K

- Sales By Store Location

```
SELECT
    Store_Location,
    concat(round(SUM(unit_price * transaction_qty)/1000,2),'K') as Total_Sales
FROM coffee_shop_sales
WHERE
    MONTH(transaction_date) =5 -- For May
GROUP BY Store_Location
ORDER BY Total_Sales DESC;
```



The screenshot shows a 'Result Grid' with two columns: 'Store_Location' and 'Total_Sales'. The data is as follows:

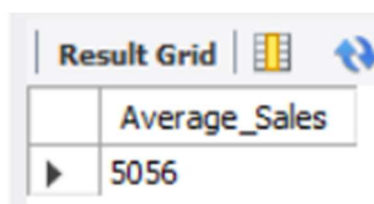
Store_Location	Total_Sales
Hell's Kitchen	52.60K
Astoria	52.43K
Lower Manhattan	51.70K

- **Sales Trend Over Period**

```
SELECT ROUND (AVG (total_sales)) AS Average_Sales
FROM (
    SELECT
        SUM (unit_price * transaction_qty) AS Total_Sales
    FROM
        coffee_shop_sales
    WHERE
        MONTH (transaction_date) = 5 -- For May
    GROUP BY
        transaction_date
) AS internal_query;
```

Query Explanation:

- This inner subquery calculates the total sales (unit_price * transaction_qty) for each date in May. It filters the data to include only transactions that occurred in May by using the MONTH() function to extract the month from the transaction_date column and filtering for May (month number 5).
- The GROUP BY clause groups the data by transaction_date, ensuring that the total sales are aggregated for each individual date in May.
- The outer query calculates the average of the total sales over all dates in May. It references the result of the inner subquery as a derived table named internal_query.
- The AVG() function calculates the average of the total_sales column from the derived table, giving us the average sales for May.
- ROUND function rounds the result to the nearest integer.



	Average_Sales
▶	5056

- **Daily Sales For Month Selected**

```

SELECT
    DAY (transaction_date) AS Day_of_Month,
    ROUND (SUM (unit_price * transaction_qty)) AS Total_Sales
FROM
    coffee_shop_sales
WHERE
    MONTH (transaction_date) = 5 -- For May
GROUP BY
    DAY (transaction_date)
ORDER BY
    DAY (transaction_date);

```

Result Grid  Filter Rows: 		
	Day_of_Month	Total_Sales
▶	1	4731
	2	4626
	3	4715
	4	4590
	5	4701
	6	4205
	7	4543
	8	5604
	9	5101
	10	5256
	11	4850
	12	4681
	13	5512
	14	5053
	15	5385
	16	5542

Result Grid  Filter Rows: 		
	Day_of_Month	Total_Sales
	17	5418
	18	5583
	19	5658
	20	5519
	21	5371
	22	5541
	23	5243
	24	5391
	25	5231
	26	5301
	27	5559
	28	4339
	29	3960
	30	4835
	31	4684

- **COMPARING DAILY SALES WITH AVERAGE SALES – IF GREATER THAN “ABOVE AVERAGE” and LESSER THAN “BELOW AVERAGE”**

```

SELECT
    Day_of_Month,
    CASE
        WHEN total_sales > avg_sales THEN 'Above Average'
        WHEN total_sales < avg_sales THEN 'Below Average'
        ELSE 'Average'
    END AS Sales_Status,
    Avg_Sales,
    Total_Sales
FROM (
    SELECT
        DAY (transaction_date) AS day_of_month,
        ROUND (SUM (unit_price * transaction_qty)) AS total_sales,
        ROUND (AVG (SUM (unit_price * transaction_qty)) OVER ()) AS avg_sales
    FROM
        coffee_shop_sales
    WHERE
        MONTH (transaction_date) = 5 -- For May
    GROUP BY
        DAY (transaction_date)
) AS sales_data
ORDER BY
    day_of_month;

```

Result Grid

Filter Rows:

Export:

	Day_of_Month	Sales_Status	Avg_Sales	Total_Sales
▶	1	Below Average	5056	4731
	2	Below Average	5056	4626
	3	Below Average	5056	4715
	4	Below Average	5056	4590
	5	Below Average	5056	4701
	6	Below Average	5056	4205
	7	Below Average	5056	4543
	8	Above Average	5056	5604
	9	Above Average	5056	5101
	10	Above Average	5056	5256
	11	Below Average	5056	4850
	12	Below Average	5056	4681
	13	Above Average	5056	5512
	14	Below Average	5056	5053
	15	Above Average	5056	5385
	16	Above Average	5056	5542

Result Grid




Filter Rows:

Export:



	Day_of_Month	Sales_Status	Avg_Sales	Total_Sales
	17	Above Average	5056	5418
	18	Above Average	5056	5583
	19	Above Average	5056	5658
	20	Above Average	5056	5519
	21	Above Average	5056	5371
	22	Above Average	5056	5541
	23	Above Average	5056	5243
	24	Above Average	5056	5391
	25	Above Average	5056	5231
	26	Above Average	5056	5301
	27	Above Average	5056	5559
	28	Below Average	5056	4339
	29	Below Average	5056	3960
	30	Below Average	5056	4835
	31	Below Average	5056	4684

- Sales By Product Category

```
SELECT
    Product_Category,
    CONCAT (ROUND (SUM (unit_price * transaction_qty)/1000,2),'K') as Total_Sales
FROM coffee_shop_sales
WHERE
    MONTH (transaction_date) = 5 -- For May
GROUP BY product_category
ORDER BY Total_Sales DESC;
```

Result Grid   Filter Rows:		
	Product_Category	Total_Sales
▶	Coffee	60.36K
	Tea	44.54K
	Bakery	18.57K
	Drinking Chocolate	16.32K
	Coffee beans	8.77K
	Branded	2.89K
	Loose Tea	2.40K
	Flavours	1.91K
	Packaged Chocolate	0.98K

- Sales By Products (Top 10)

```
SELECT
    Product_Type,
    CONCAT(ROUND(SUM(unit_price * transaction_qty)/1000,2),'K') as Total_Sales
FROM coffee_shop_sales
WHERE
    MONTH(transaction_date) = 5 -- For May
GROUP BY product_type
ORDER BY SUM(unit_price * transaction_qty) DESC
LIMIT 10;
```

Result Grid   Filter Rows:		
	Product_Type	Total_Sales
▶	Barista Espresso	20.42K
	Brewed Chai tea	17.43K
	Hot chocolate	16.32K
	Gourmet brewed coffee	15.56K
	Brewed herbal tea	10.93K
	Brewed Black tea	10.78K
	Premium brewed coffee	8.74K
	Organic brewed coffee	8.35K
	Scone	8.31K
	Drip coffee	7.29K

- Sales By Day | Hour

```

SELECT
    ROUND(SUM(unit_price * transaction_qty)) AS Total_Sales,
    COUNT(*) AS Total_Orders,
    SUM(transaction_qty) AS Total_Quantity
FROM
    coffee_shop_sales
WHERE
    DAYOFWEEK(transaction_date) = 2 -- For Tuesday (1 is Sunday, 2 is Monday, ..., 7 is Saturday)
    AND HOUR(transaction_time) = 8 -- For hour number 8
    AND MONTH(transaction_date) = 5; -- For May
  
```

Result Grid			
Filter Rows:			
	Total_Sales	Total_Orders	Total_Quantity
▶	2697	572	819

- To Get Sales For All Hours For Month

```

SELECT
    HOUR(transaction_time) AS Hour_of_Day,
    ROUND(SUM(unit_price * transaction_qty)) AS Total_Sales
FROM
    coffee_shop_sales
WHERE
    MONTH(transaction_date) = 5 -- For May
GROUP BY
    HOUR(transaction_time)
ORDER BY
    HOUR(transaction_time);
  
```

Result Grid		
Filter Rows:		
	Hour_of_Day	Total_Sales
▶	6	4913
	7	14351
	8	18822
	9	19145
	10	19639
	11	10312
	12	8870
	13	9379
	14	9058
	15	9525
	16	9154
	17	8967
	18	7680
	19	6256
	20	656

- To Get Sales From Monday To Sunday For Month

```

SELECT
CASE
    WHEN DAYOFWEEK(transaction_date) = 2 THEN 'Monday'
    WHEN DAYOFWEEK(transaction_date) = 3 THEN 'Tuesday'
    WHEN DAYOFWEEK(transaction_date) = 4 THEN 'Wednesday'
    WHEN DAYOFWEEK(transaction_date) = 5 THEN 'Thursday'
    WHEN DAYOFWEEK(transaction_date) = 6 THEN 'Friday'
    WHEN DAYOFWEEK(transaction_date) = 7 THEN 'Saturday'
    ELSE 'Sunday'
END AS Day_of_Week,
CONCAT(ROUND(SUM(unit_price * transaction_qty)/1000),'K') AS Total_Sales
FROM
    coffee_shop_sales
WHERE
    MONTH(transaction_date) = 5 -- For May
GROUP BY
CASE
    WHEN DAYOFWEEK(transaction_date) = 2 THEN 'Monday'
    WHEN DAYOFWEEK(transaction_date) = 3 THEN 'Tuesday'
    WHEN DAYOFWEEK(transaction_date) = 4 THEN 'Wednesday'
    WHEN DAYOFWEEK(transaction_date) = 5 THEN 'Thursday'
    WHEN DAYOFWEEK(transaction_date) = 6 THEN 'Friday'
    WHEN DAYOFWEEK(transaction_date) = 7 THEN 'Saturday'
    ELSE 'Sunday'
END;

```

Result Grid			Filter Rows:
	Day_of_Week	Total_Sales	
▶	Monday	25221	
	Tuesday	25347	
	Wednesday	25465	
	Thursday	20254	
	Friday	20341	
	Saturday	20795	
	Sunday	19305	

